```
  redis:
    image: redis

  database:
    image: postgres
    env_file:
      - .env/development/database
    volumes:
      - db_data:/var/lib/postgresql/data

  selenium_chrome:
    image: selenium/standalone-chrome-debug
    logging:
      driver: none
    ports:
      - "5900:5900"

volumes:
  db_data:
➤ gem_cache:
```

Just like we did in Decoupling Data from the Container, on page 81, we create a new named volume—this time called gem_cache—by adding this to our list of volumes. Compose will handle the details of where this volume is stored.

Then, in the definition for our web service, we tell Compose to mount our gem_cache volume at /gems in the container, which is now where Bundler is configured to install gems to.

To try out this approach, we first need to rebuild our image:

```
$ docker-compose build web
```

Our web service should already be stopped, and we removed its container earlier, so to create a new web container along with the gem_cache volume, we just do:

```
$ docker-compose up -d web
Creating volume "myapp_gem_cache" with default driver
Recreating myapp_web_1 ... done
```

With the app running, and our gem_cache created, we can now execute Bundler commands directly against our running web container:

```
$ docker-compose exec web bundle install
```

Thanks to our BUNDLE_PATH, this installs the projects' gems to the container's /gems directory. As we know, that's where our gem_cache volume is mounted, courtesy of our volume mapping. As a consequence, all our gems are now installed on our gem_cache volume.